



<https://algs4.cs.princeton.edu>

2.4 PRIORITY QUEUES

- *APIs*
- *elementary implementations*
- *binary heaps*
- *heapsort*

insert：新节点放在叶子节点上（注意保持完全二叉树的结构），然后swim：和父节点比较，如果不符合，就exch【复杂度 $1 \times \text{层数} = \log n$ 】

delm：将最后的节点放到根节点上，然后sink：和所有叶子节点比较，找到叶子节点的最值，放到父节点的位置【这个就决定了del不是stable的】
【复杂度 $d \times \text{层数} = d \log n$ （其中d就是一个父节点有几个孩子节点）】

初始化pq：什么元素都没有，一个空数组



<https://algs4.cs.princeton.edu>

2.4 PRIORITY QUEUES

- *APIs*
- *elementary implementations*
- *binary heaps*
- *heapsort*

Collections

A **collection** is a data type that stores a group of items.

data type	core operations	data structure
stack	PUSH, POP	<i>singly linked list</i> <i>resizable array</i>
queue	ENQUEUE, DEQUEUE	
deque	ADD-FIRST, REMOVE-FIRST, ADD-LAST, REMOVE-LAST	<i>doubly linked list</i> <i>resizable array</i>
priority queue	INSERT, DELETE-MAX	<i>binary heap</i>
symbol table	PUT, GET, DELETE	<i>binary search tree</i> <i>hash table</i>
set	ADD, CONTAINS, DELETE	

数据类型是抽象
数据结构是实现

Priority queue

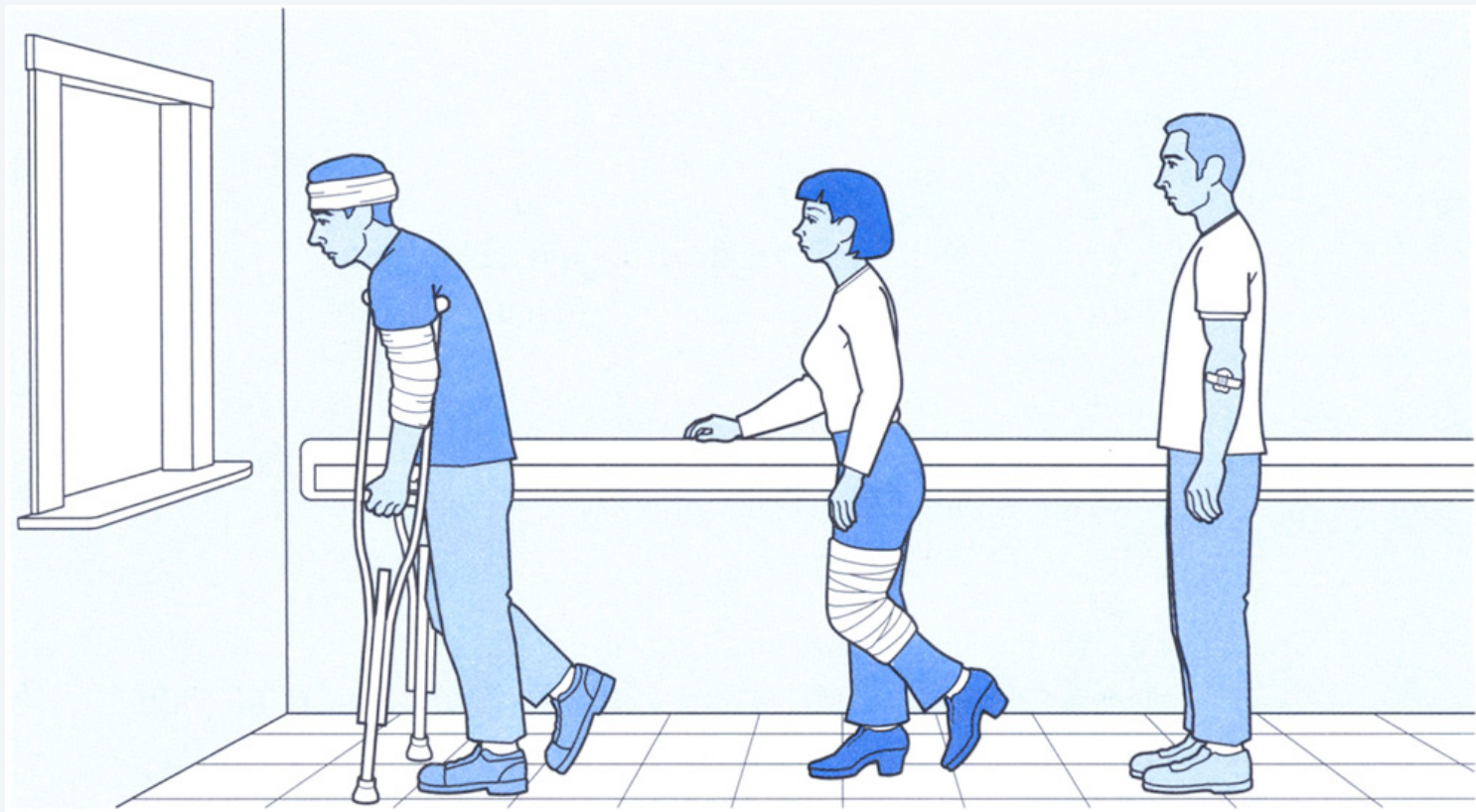
Collections. Insert and remove items. Which item to remove?

Stack. Remove the item most recently added.

Queue. Remove the item least recently added.

Randomized queue. Remove a random item.

Priority queue. Remove the **largest** (or **smallest**) item.



triage in an emergency room
(priority = urgency of wound/illness)

operation	argument	return value
insert	P	
insert	Q	
insert	E	
remove max		Q
insert	X	
insert	A	
insert	M	
remove max		X
insert	P	
insert	L	
insert	E	
remove max		P

Max-oriented priority queue API

“bounded type parameter”
↙
`public class MaxPQ<Key> extends Comparable<Key>>`

	<code>MaxPQ()</code>	<i>create an empty priority queue</i>
<code>void</code>	<code>insert(Key key)</code>	<i>insert a key</i>
<code>Key</code>	<code>delMax()</code>	<i>return and remove a largest key</i>
<code>Key</code>	<code>max()</code>	<i>return a largest key</i>
<code>boolean</code>	<code>isEmpty()</code>	<i>is the priority queue empty?</i>
<code>int</code>	<code>size()</code>	<i>number of keys in the priority queue</i>

Note 1. Keys are generic, but must be `Comparable`.

Note 2. Duplicate keys allowed; `delMax()` removes and returns any maximum key.

Min-oriented priority queue API

Analogous to [MaxPQ](#).

```
public class MinPQ<Key extends Comparable<Key>>
```

	MinPQ()	<i>create an empty priority queue</i>
--	---------	---------------------------------------

void	insert(Key key)	<i>insert a key</i>
------	-----------------	---------------------

Key	delMin()	<i>return and remove a smallest key</i>
-----	----------	---

Key	min()	<i>return a smallest key</i>
-----	-------	------------------------------

boolean	isEmpty()	<i>is the priority queue empty?</i>
---------	-----------	-------------------------------------

int	size()	<i>number of keys in the priority queue</i>
-----	--------	---

[Warmup client](#). Sort a stream of integers from standard input.

Priority queue: applications

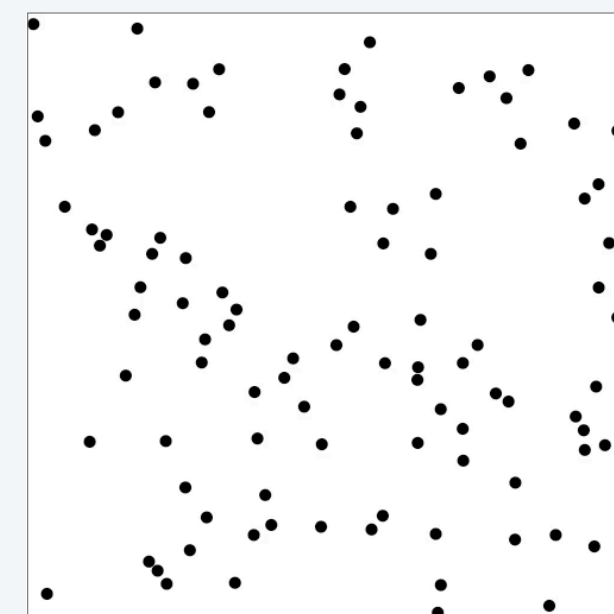
- Event-driven simulation. [customers in a line, colliding particles]
- Discrete optimization. [bin packing, scheduling]
- Artificial intelligence. [A* search]
- Computer networks. [web cache]
- Data compression. [Huffman codes]
- Operating systems. [load balancing, interrupt handling]
- Graph searching. [Dijkstra's algorithm, Prim's algorithm]
- Number theory. [sum of powers]
- Spam filtering. [Bayesian spam filter]
- Statistics. [online median in data stream]



priority = length of
best known path

8	4	7
1	5	6
3	2	

priority = "distance"
to goal board



priority = event time



<https://algs4.cs.princeton.edu>

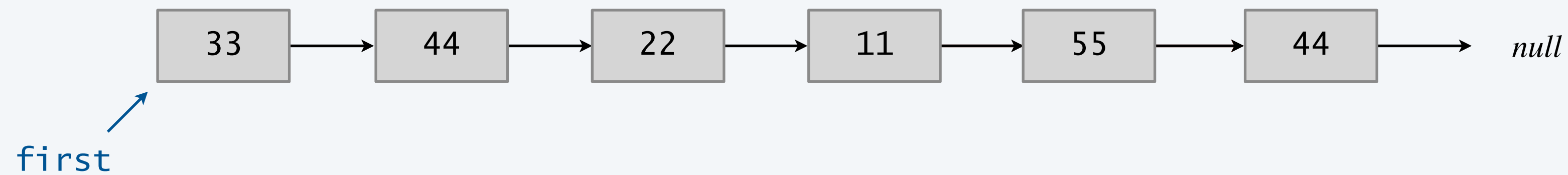
2.4 PRIORITY QUEUES

- *APIs*
- *elementary implementations*
- *binary heaps*
- *heapsort*

Priority queue: elementary implementations

Unordered list. Store keys in a singly linked list.

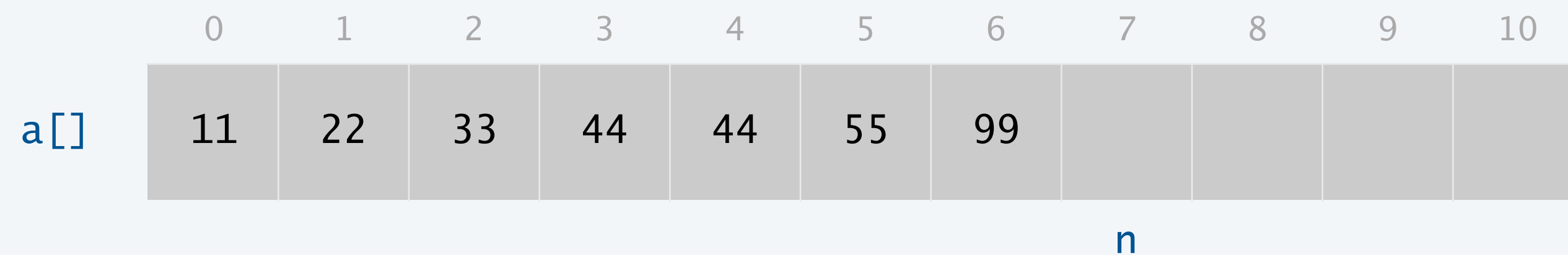
list : 链表
array : 数组



Performance. INSERT takes $\Theta(1)$ time; DELETE-MAX takes $\Theta(n)$ time.

Priority queue: elementary implementations

Ordered array. Store keys in an array in ascending (or descending) order.



ordered array implementation of a MaxPQ



What are the worst-case running times for INSERT and DELETE-MAX, respectively, in a MaxPQ implemented with an **ordered array** ?

ignore array resizing

- A. $\Theta(1)$ and $\Theta(n)$
- B. $\Theta(1)$ and $\Theta(\log n)$
- C. $\Theta(\log n)$ and $\Theta(1)$
- ☒ D. $\Theta(n)$ and $\Theta(1)$

误：数组的插入操作分成两步：找到位置&腾出位置
腾出位置的最坏时间复杂度始终是n（不断移动元素）



ordered array implementation of a MaxPQ

Priority queue: implementations cost summary

Elementary implementations. Either INSERT or DELETE-MAX takes $\Theta(n)$ time.

implementation	INSERT	DELETE-MAX
unordered list	1	n
ordered array	n	1
goal	$\log n$	$\log n$

order of growth of running time for priority queue with n items

Challenge. Implement both INSERT and DELETE-MAX efficiently.

Solution. “Somewhat-ordered” array.



<https://algs4.cs.princeton.edu>

2.4 PRIORITY QUEUES

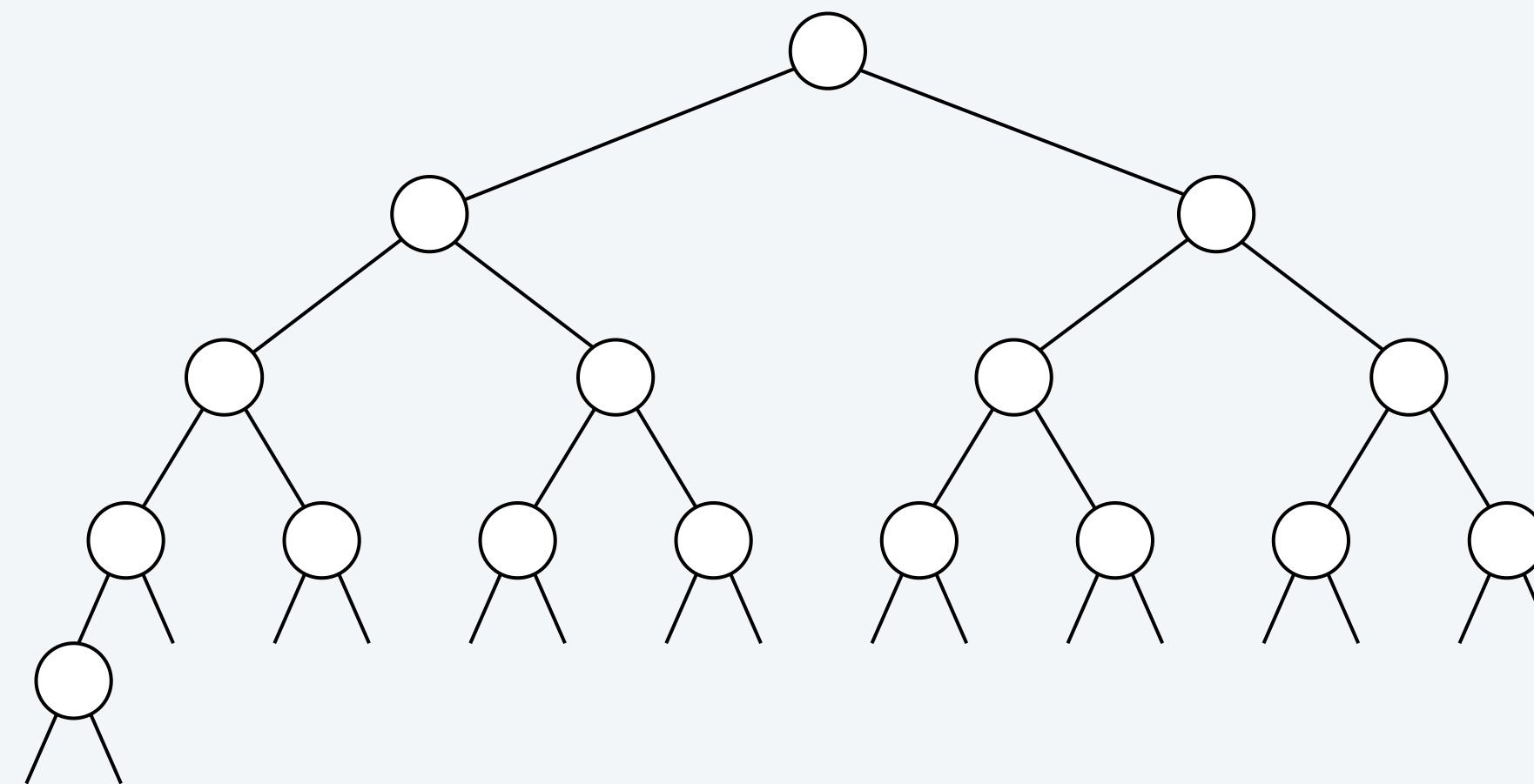
- *APIs*
- *elementary implementations*
- *binary heaps*
- *heapsort*

Complete binary tree

Binary tree. Empty **or** node with links to two disjoint binary trees (left and right subtrees).

Complete tree. Every level (except possibly the last) is completely filled; the last level is filled from left to right.

完全二叉树 (Complete Binary Tree)
除了最后一层，其他所有层的节点都被完全填满；
最后一层的节点全部靠左对齐，从左到右依次填充，不能有空缺。



complete binary tree
($n = 16$ nodes, height = 4)

Property. Height of complete binary tree with n nodes is $\lfloor \log_2 n \rfloor$.

Pf. As you successively add nodes, height increases (by 1) only when n is a power of 2.

A complete binary tree in nature (of height 4)



Hyphaene Compressa - Doum Palm

© Shlomit Pinter

Binary heap: representation

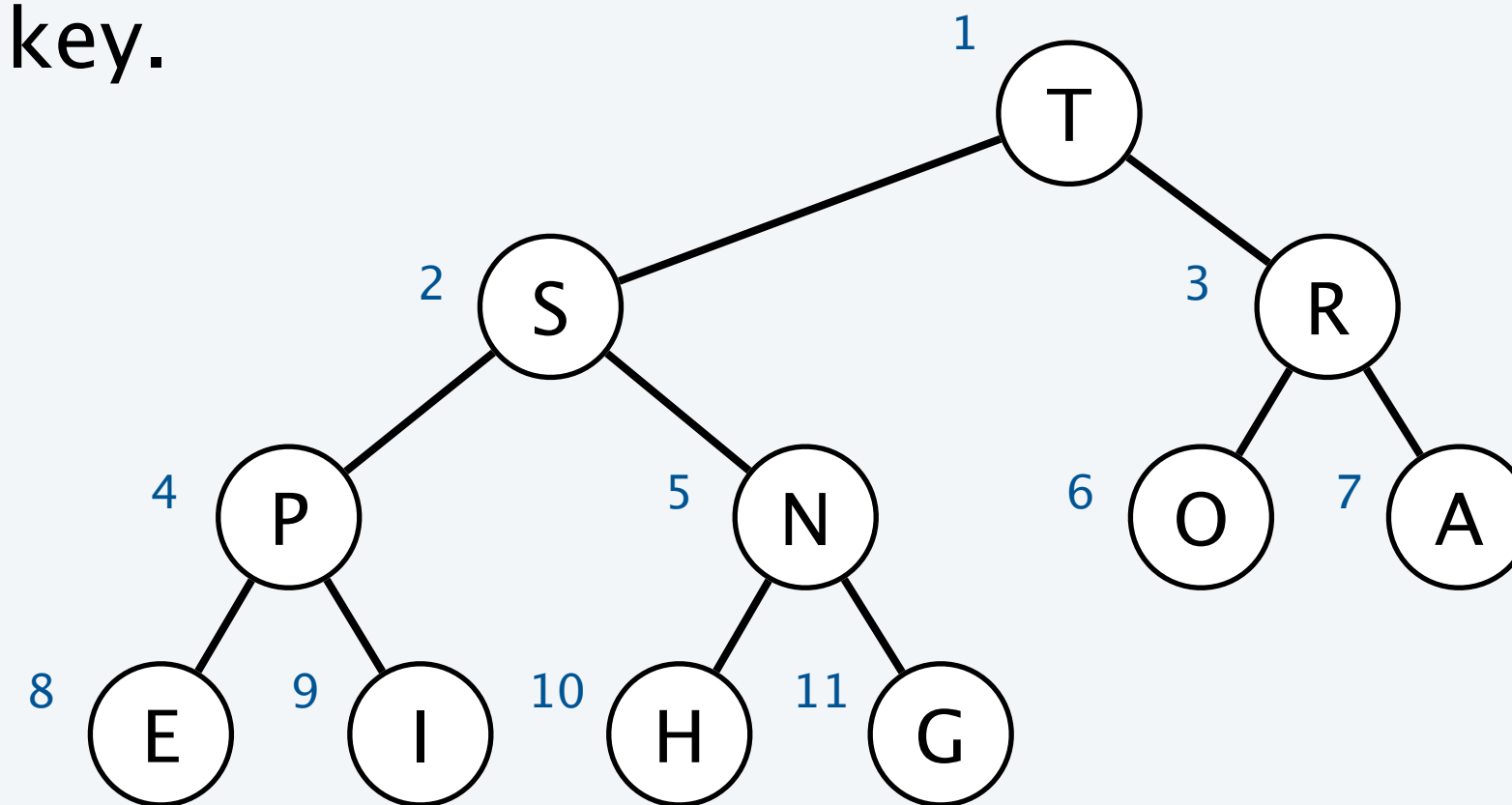
Binary heap. Array representation of a heap-ordered complete binary tree.

Heap-ordered tree.

- Keys in nodes.
- Child's key no larger than parent's key.

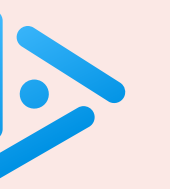
Array representation.

- Indices start at 1.
- Take nodes in **level order**.
- No explicit links!



explicit: 显式的 (比如具体的指针数据结构的实现)
implicit: 隐含的

	0	1	2	3	4	5	6	7	8	9	10	11
a[]	-	T	S	R	P	N	O	A	E	I	H	G



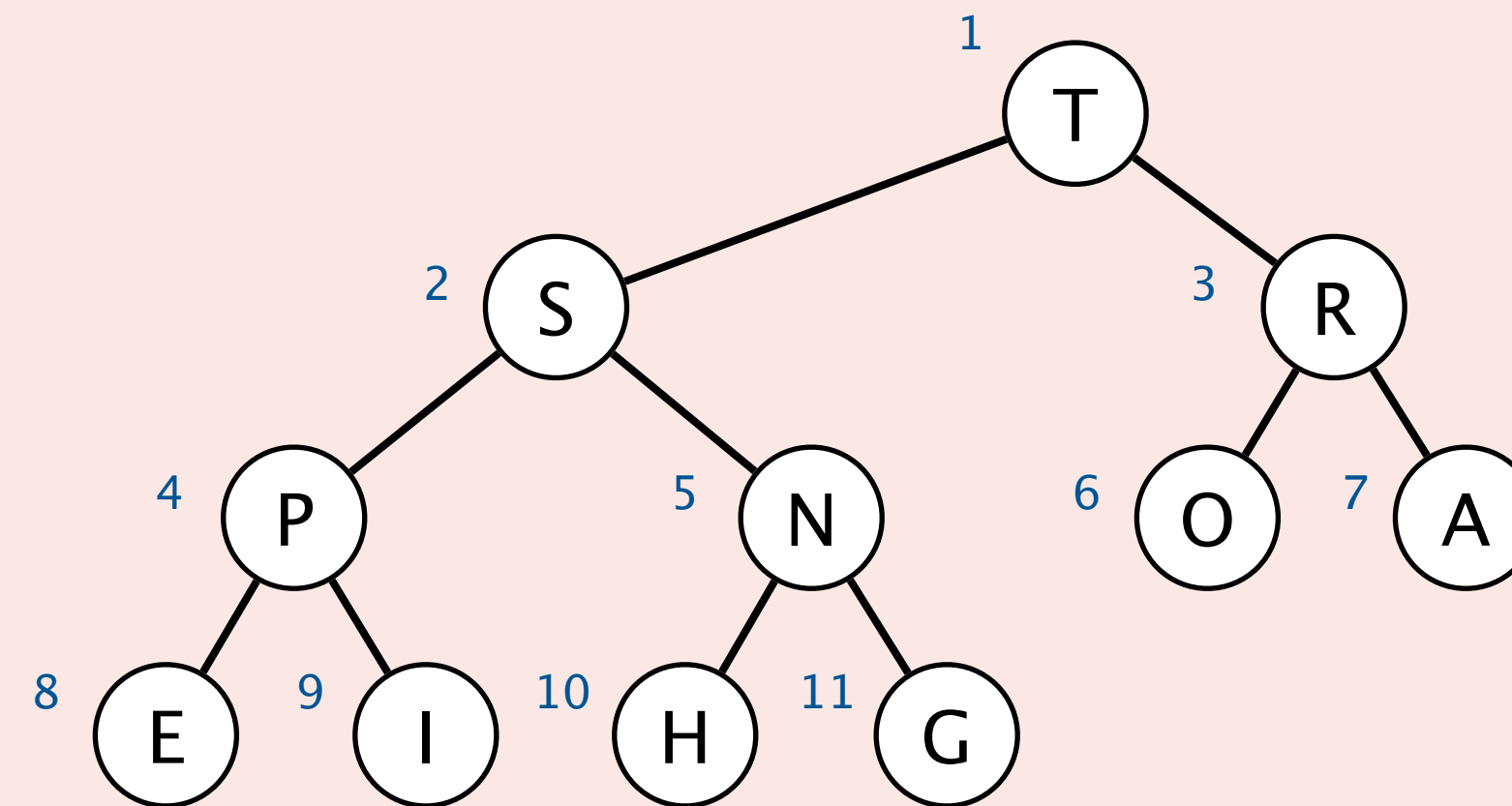
Consider the node at index k in a binary heap. Which Java expression produces the index of its parent?

A. $(k - 1) / 2$

B. $k / 2$

C. $(k + 1) / 2$

D. $2 * k$



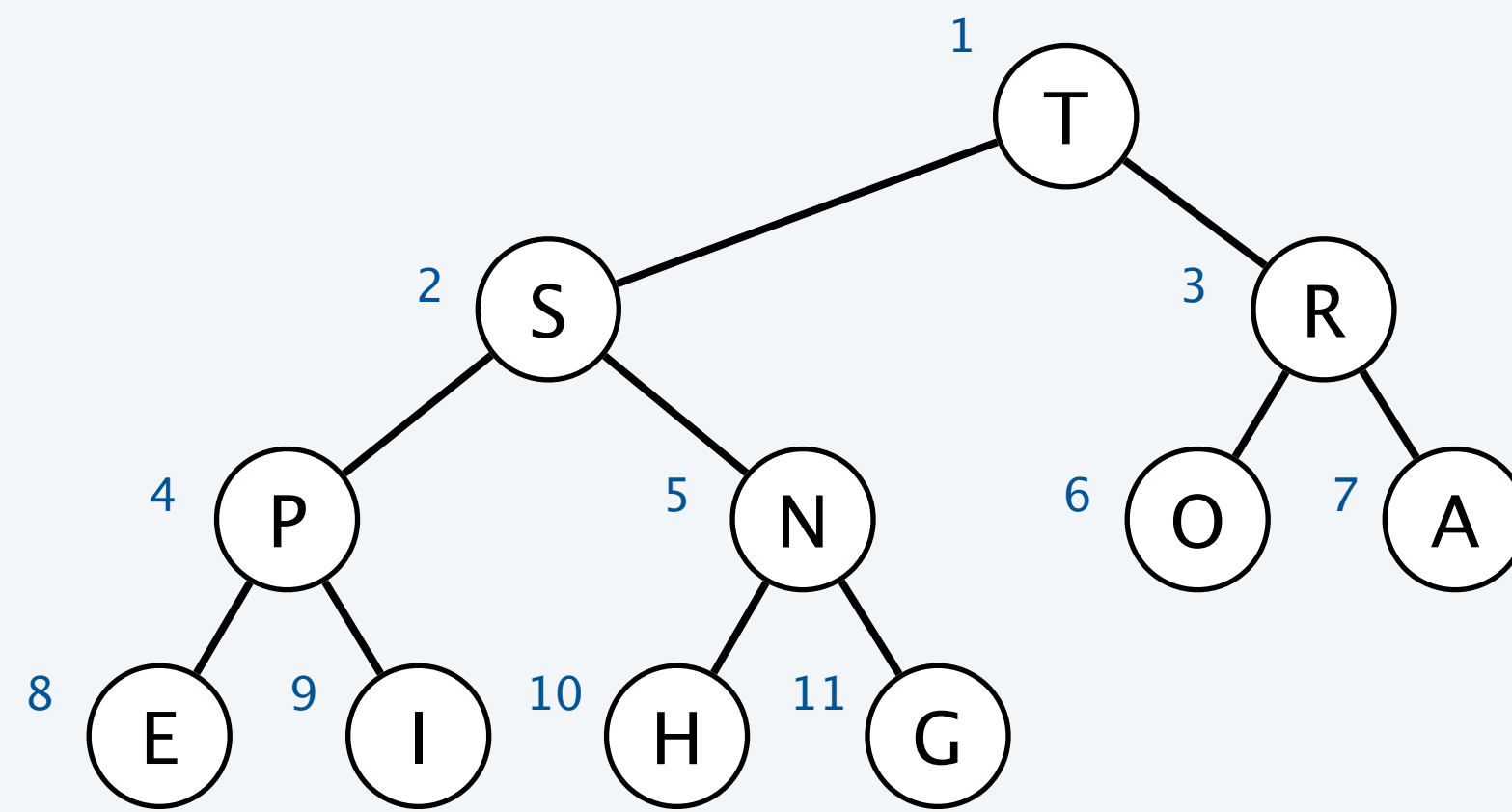
	0	1	2	3	4	5	6	7	8	9	10	11
a[]	-	T	S	R	P	N	O	A	E	I	H	G

Binary heap: properties

Proposition. Largest key is at index 1, which is root of binary tree.

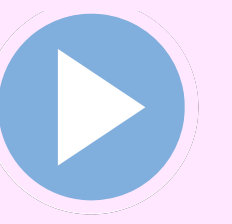
Proposition. Can use array indices to move up or down tree.

- Parent of key at index k is at index $k/2$.
- Children of key at index k are at indices $2*k$ and $2*k + 1$.



	0	1	2	3	4	5	6	7	8	9	10	11
a[]	-	T	S	R	P	N	O	A	E	I	H	G

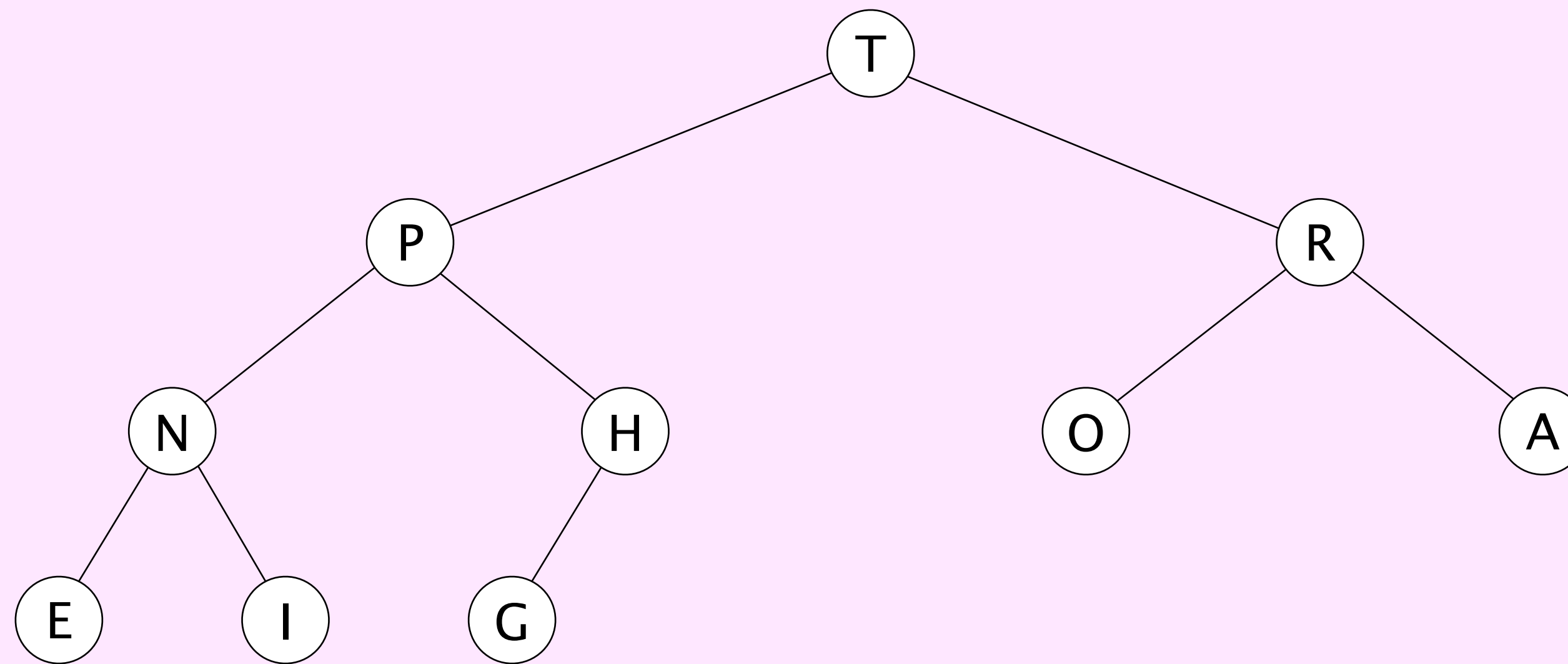
根节点下标是1 !!
下标0处为空



Insert. Add node at end, then **swim** it up.

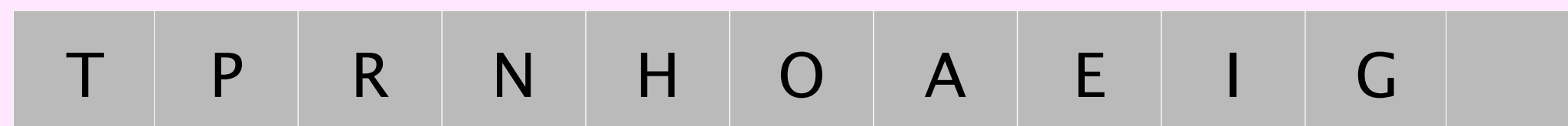
Remove the maximum. Exchange root with node at end, then **sink** it down.

heap ordered



大/小顶堆（二叉堆）完全不要求左右子
节点之间有任何大小关系！！
只要求父与子的大小关系

一开始插入到数组的尾部
排序的根本目标是保持父子间的有序关系



Binary heap: promotion

Scenario. Key in node becomes **larger** than key in parent's node.

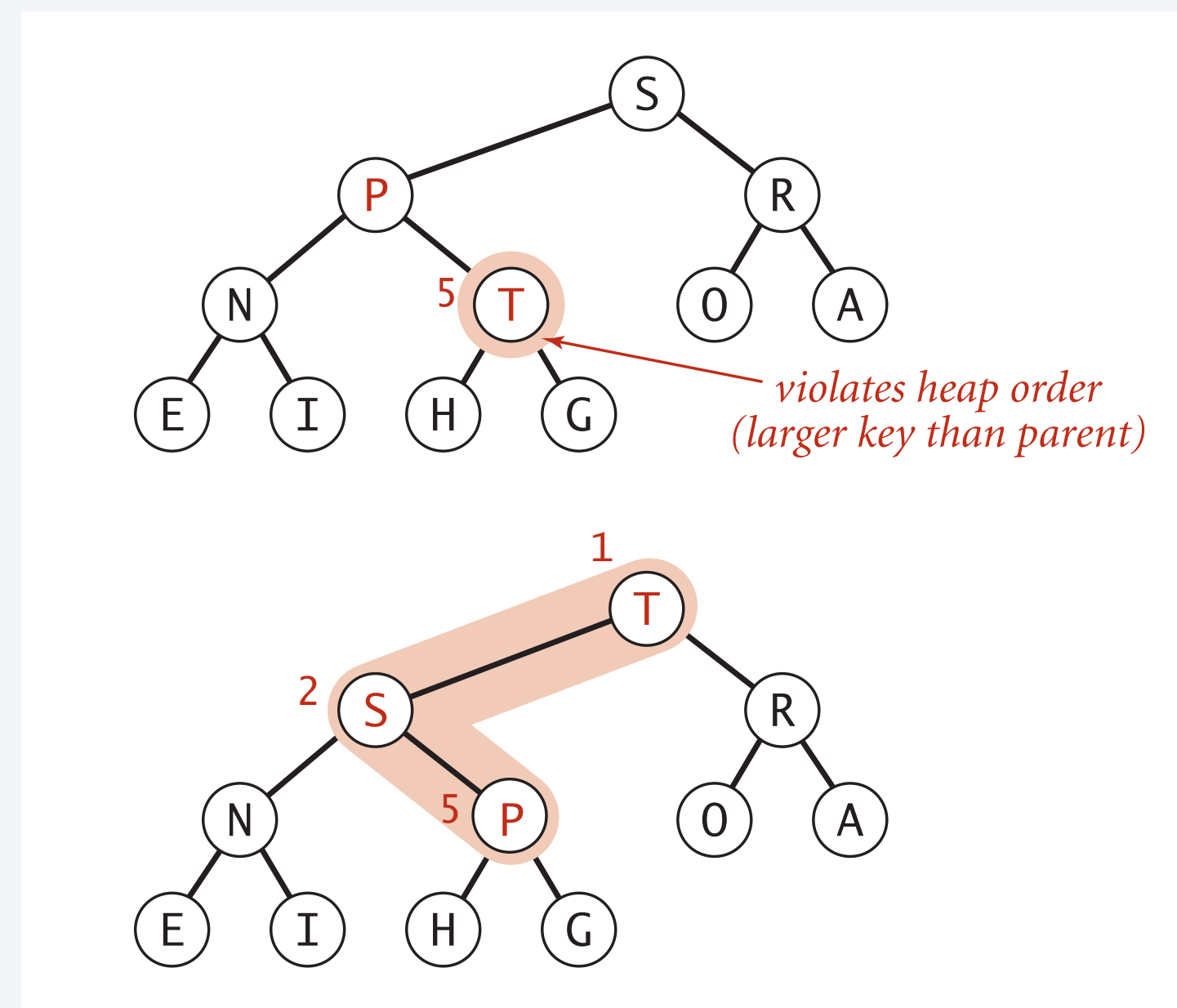
To eliminate the violation:

- Exchange key in child node with key in parent node.
- Repeat until heap order restored.

```
private void swim(int k) {  
    while (k > 1 && less(k/2, k)) {  
        exch(k, k/2);  
        k = k/2;  
    }  
}
```

parent of node at k is at k/2

父与子交换，并重复这个过程



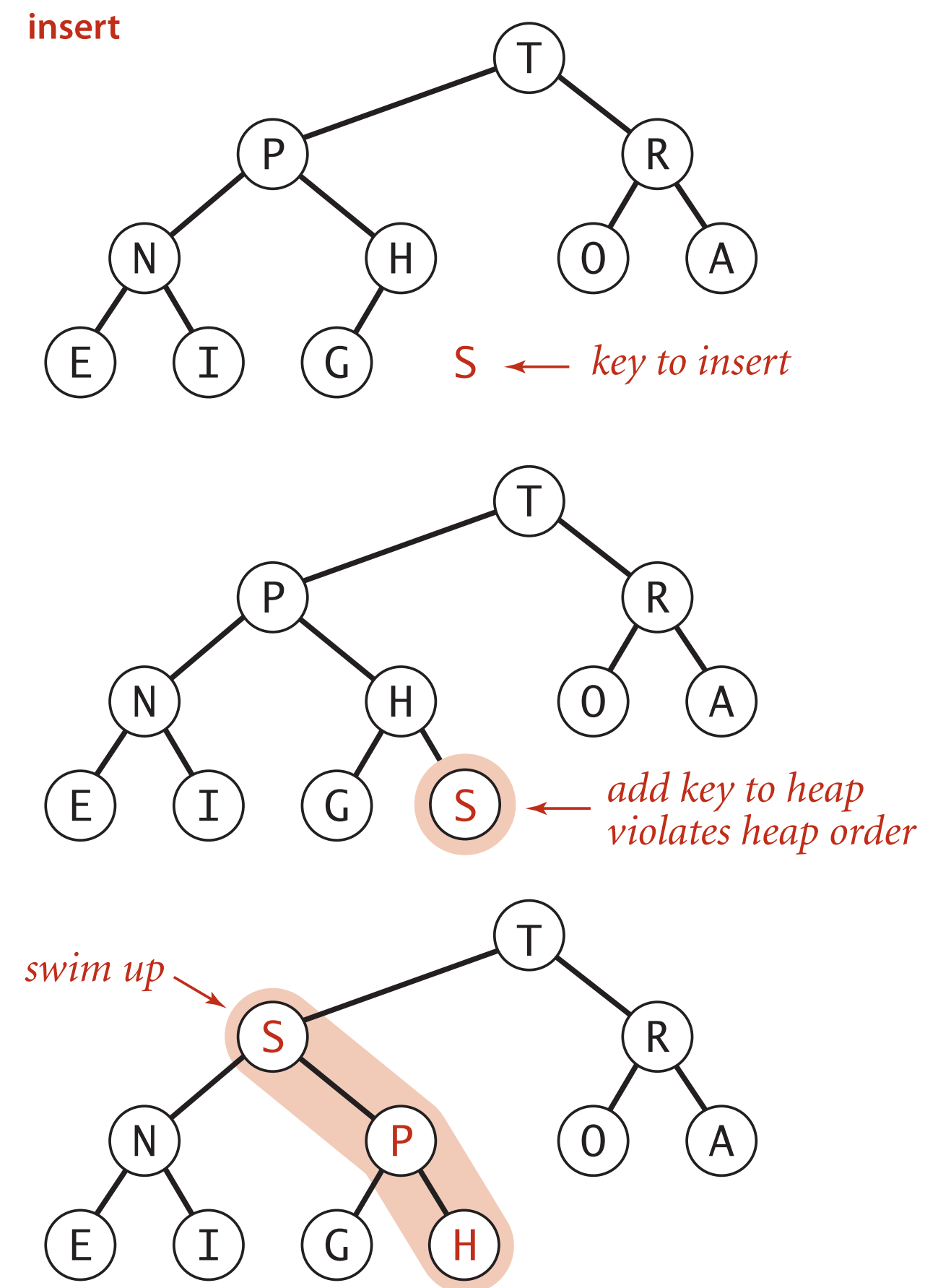
Peter principle. Node promoted to level of incompetence.

Binary heap: insertion

Insert. Add node at end in bottom level; then, swim it up.

Cost. At most $1 + \log_2 n$ compares.

```
public void insert(Key x) {  
    pq[++n] = x;  
    swim(n);  
}
```



Binary heap: demotion

Scenario. Key in node becomes **smaller** than one (or both) of keys in childrens' nodes.

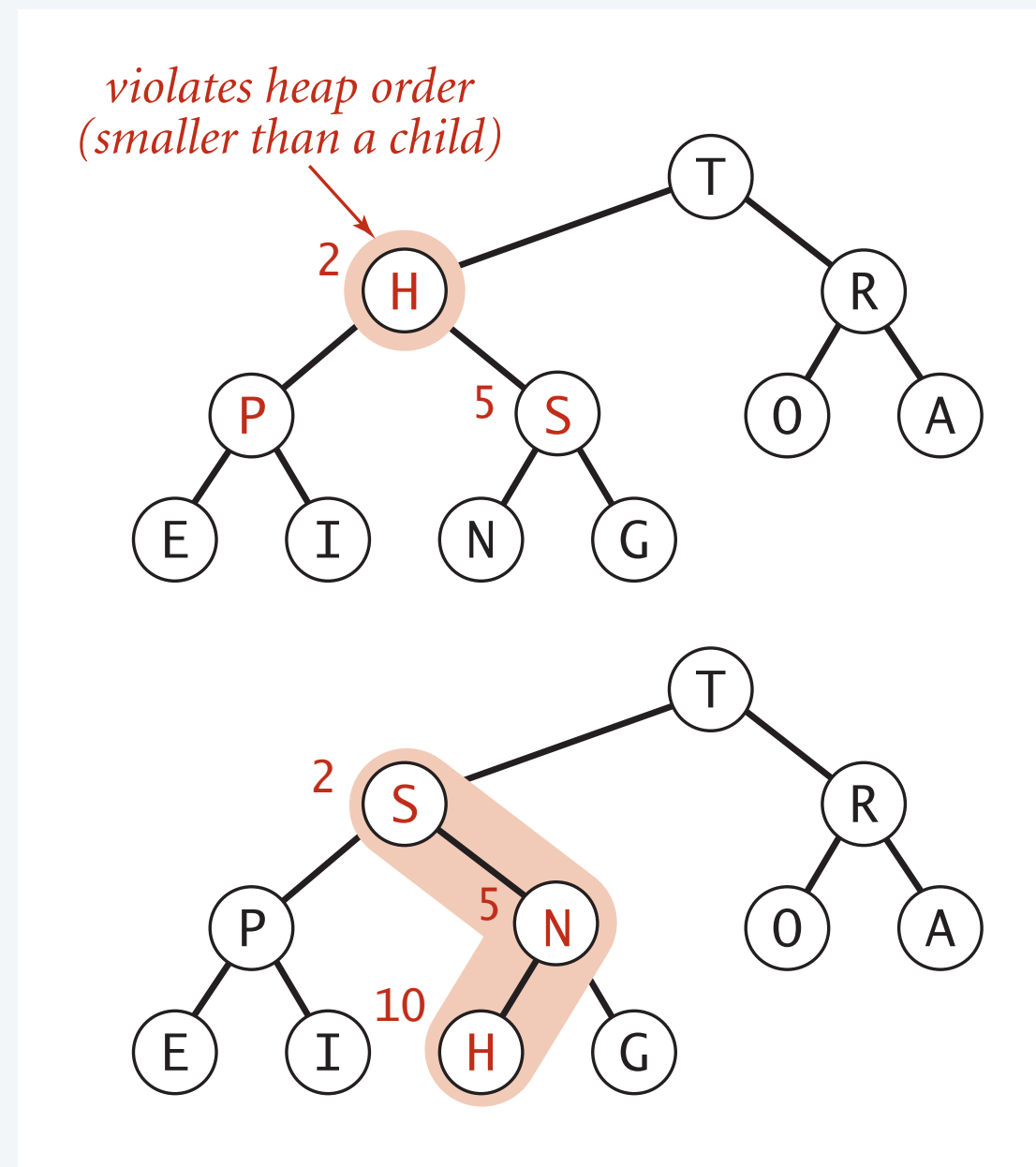
To eliminate the violation:

why not smaller child ?

- Exchange key in parent node with key in larger child's node.
- Repeat until heap order restored.

```
private void sink(int k) {  
    while (2*k <= n) {  
        int j = 2*k;  
        if (j < n && less(j, j+1))  
            j++;  
        if (!less(k, j)) break;  
        exch(k, j);  
        k = j;  
    }  
}
```

*children of node at k
are at 2*k and 2*k+1*



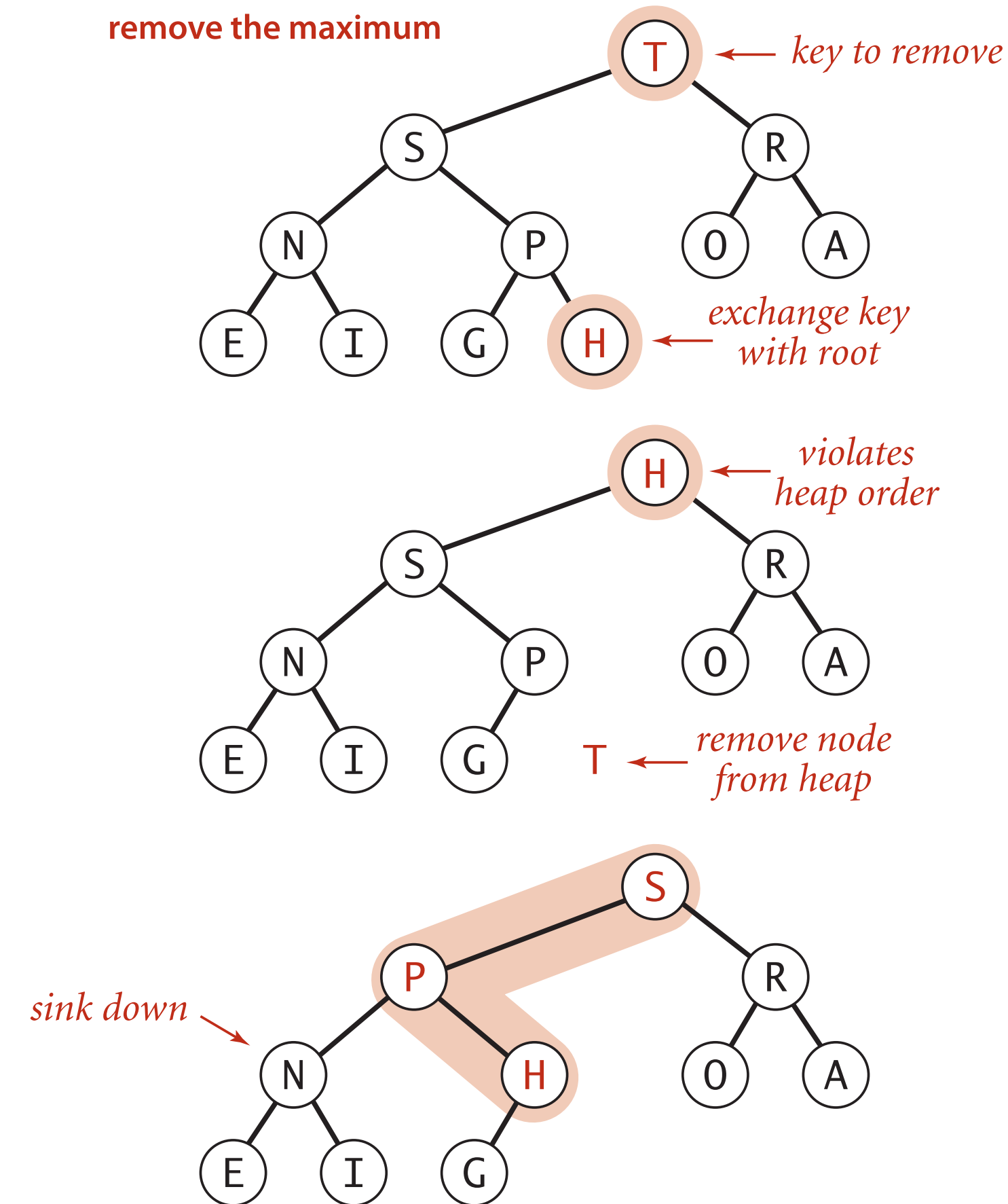
Power struggle. Better subordinate promoted.

Binary heap: delete the maximum

Delete max. Exchange root with node at end; then, sink it down.

Cost. At most $2 \log_2 n$ compares.

```
public Key delMax() {  
    Key max = pq[1];  
    exch(1, n--);  
    sink(1);  
    pq[n+1] = null; ← prevent loitering  
    return max;  
}
```



Binary heap: Java implementation

```
public class MaxPQ<Key extends Comparable<Key>> {  
    private Key[] a;  
    private int n;
```

```
    public MaxPQ(int capacity) {  
        a = (Key[]) new Comparable[capacity+1];  
    }
```

```
    public void insert(Key key) // see previous code  
    public Key delMax()        // see previous code
```

```
    private void swim(int k) // see previous code  
    private void sink(int k) // see previous code
```

```
    private boolean less(int i, int j) {  
        return a[i].compareTo(a[j]) < 0;  
    }
```

```
    private void exch(int i, int j)  
    { Key temp = a[i]; a[i] = a[j]; a[j] = temp; }
```

```
}
```

*fixed capacity
(for simplicity)*

PQ ops

heap helper functions

array helper functions

```
public void insert(Key key) {  
    a[++n] = key; // 新元素放到  
    数组末尾 (堆底)  
    swim(n); // 上浮到正确  
    位置, 恢复堆序  
}
```

```
public Key delMax() {  
    Key max = a[1]; // 根节点  
    是全局最大值  
    exch(1, n); // 交换根节  
    点与堆底元素  
    a[n--] = null; // 删除堆底  
    元素, 帮助GC  
    sink(1); // 下沉根节点  
    恢复堆序  
    return max;  
}
```


Priority queue: implementations cost summary

Goal. Implement both INSERT and DELETE-MAX in $\Theta(\log n)$ time.

implementation	INSERT	DELETE-MAX	MAX
unordered list	1	n	n
ordered array	n	1	1
goal	$\log n$	$\log n$	1

order of growth of running time for priority queue with n items

Binary heap: considerations

Underflow and overflow.

- Underflow: throw exception if deleting from empty PQ.
- Overflow: add no-arg constructor and use resizing array.

下溢 (Underflow) : 从空优先队列中执行删除操作
【处理方案: 抛出异常】
上溢 (Overflow) : 固定容量数组无法容纳更多元素
，导致插入失败 【处理方案: 使用动态扩容数组】

Minimum-oriented priority queue.

- Replace `less()` with `greater()`.
- Implement `greater()`.

最小优先队列实现: 将`less()`方法替换为`greater()`，即仅反转比较方向即可

*leads to $O(\log n)$ amortized time per op
(how to make worst case?)*

Other operations.

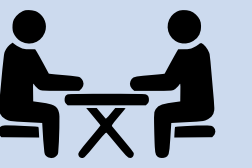
- Remove an arbitrary item.
- Change the priority of an item.

*can implement efficiently with `sink()` and `swim()`
[stay tuned for Prim/Dijkstra]*

Immutability of keys.

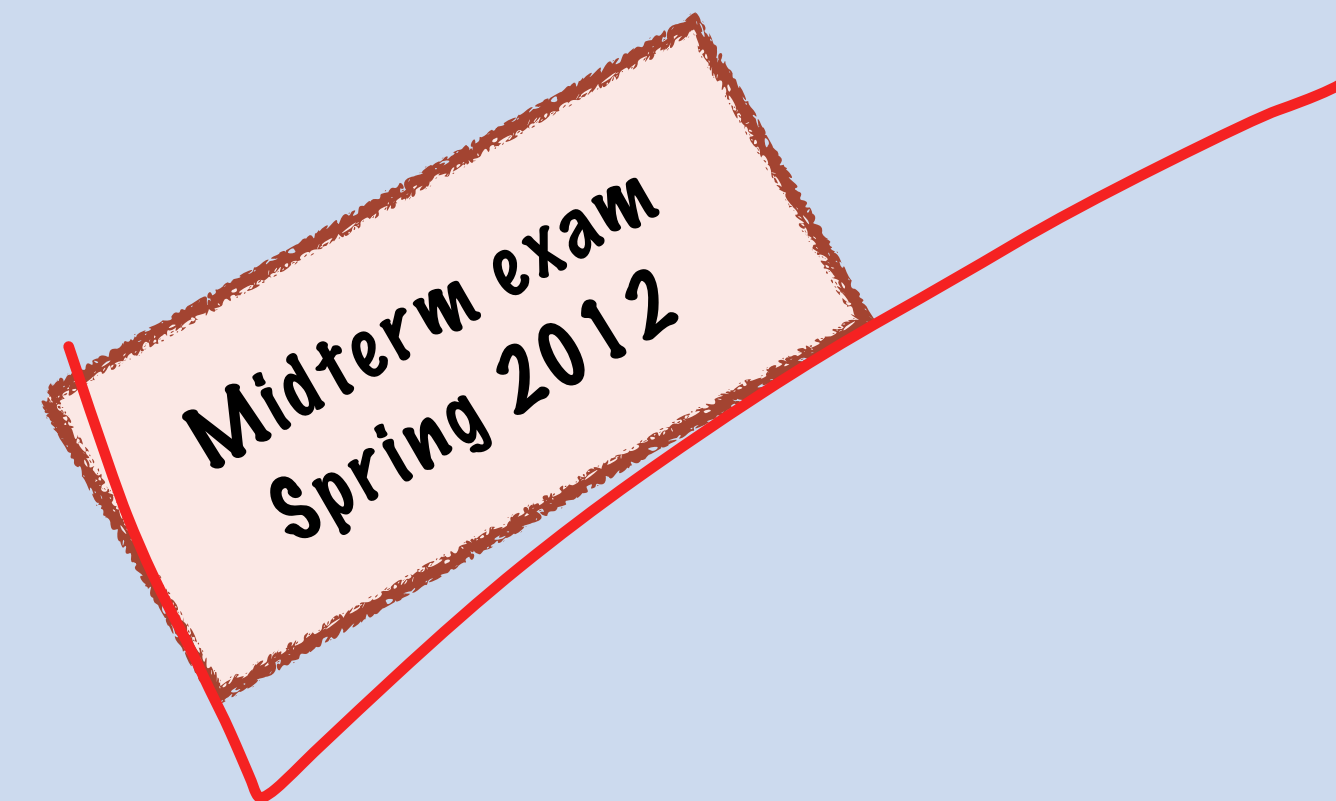
- Assumption: client does not change keys while they're on the PQ.
- Best practice: use immutable keys.

immutable in Java: String, Integer, Double, ...



Goal. Design an efficient data structure to support the following API:

- **INSERT:** insert a key.
- **DELETE-MAX:** return and remove a largest key.
- **SAMPLE:** return a random key.
- **DELETE-RANDOM:** return and remove a random key.



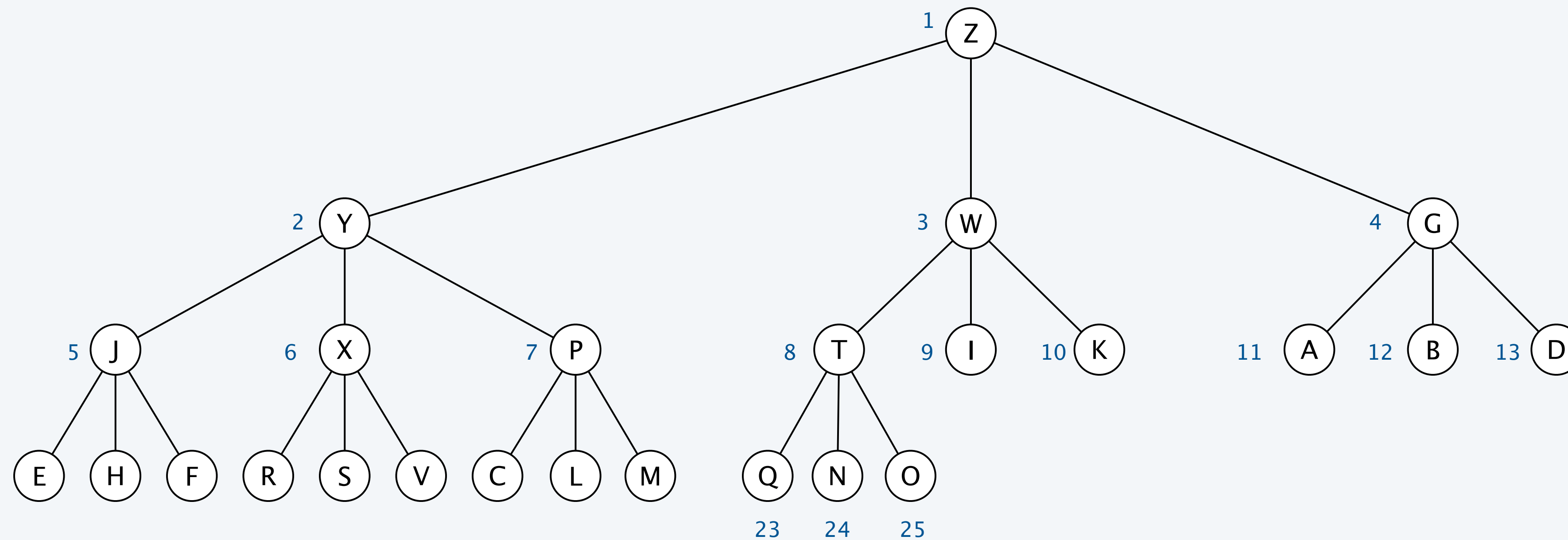
Multiway heaps

Multiway heaps.

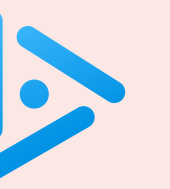
- Complete d -way tree.
- Child's key no larger than parent's key.

Property. Height of complete d -way tree on n nodes is $\sim \log_d n$.

Property. Children of key at index k at indices $3k - 1$, $3k$, and $3k + 1$; parent at $\lfloor (k + 1) / 3 \rfloor$.



3-way heap



H In the worst case, how many compares to INSERT and DELETE-MAX
in a d -way heap as function of both n and d ?

分析思路：每层最多比较几次，最多要多少层

insert：每层和父比较一次，一共 $\log n$ 层

delete：每层最多和所有 d 个子节点比较找出最大值，一共 $\log n$ 层，所以 $d \log n$

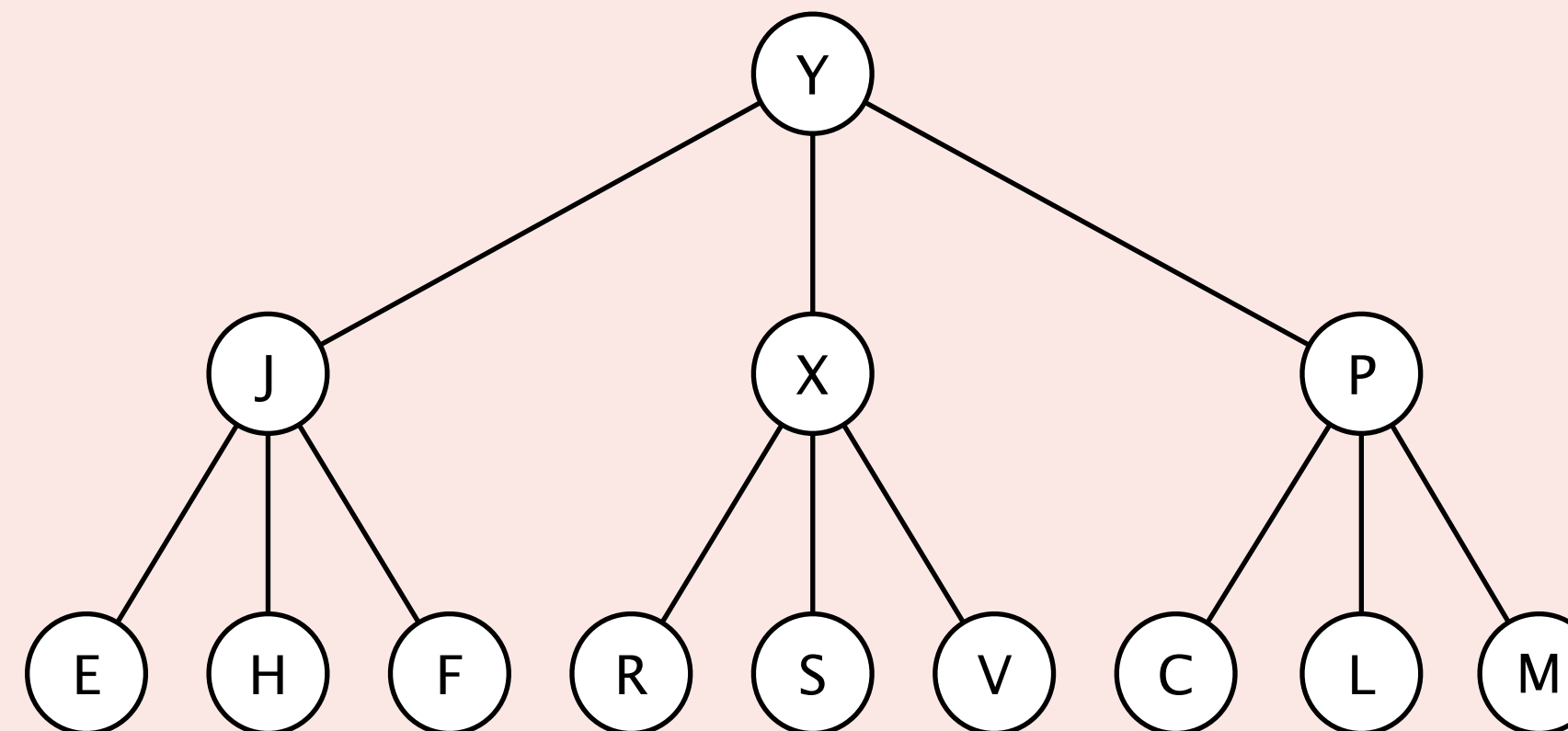
选B

A. $\sim \log_d n$ and $\sim \log_d n$

B. $\sim \log_d n$ and $\sim d \log_d n$

C. $\sim d \log_d n$ and $\sim \log_d n$

D. $\sim d \log_d n$ and $\sim d \log_d n$



Priority queue: implementation cost summary

implementation	INSERT	DELETE-MAX	MAX	
unordered list	1	n	n	
ordered array	n	1	1	
binary heap	$\log n$	$\log n$	1	
d-ary heap	$\log_d n$	$d \log_d n$	1	← <i>sweet spot: $d = 4$</i>
Fibonacci	1	$\log n$	1	← <i>see COS 423</i>
impossible	1	1	1	← <i>why impossible ?</i>

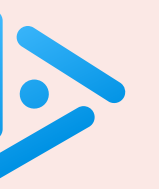
order-of-growth of running time for priority queue with n items



<https://algs4.cs.princeton.edu>

2.4 PRIORITY QUEUES

- *APIs*
- *elementary implementations*
- *binary heaps*
- *heapsort*



What are the properties of this sorting algorithm?

```
public void sort(String[] a) {  
    int n = a.length;  
    MinPQ<String> pq = new MinPQ<String>();  
  
    for (int i = 0; i < n; i++)  
        pq.insert(a[i]);  
  
    for (int i = 0; i < n; i++)  
        a[i] = pq.delMin();  
}
```

注意！首先明确n在这个情景下是什么；a.size()
每次插入是 $\log n$ ，删除min是 $2\log n$

A. $\Theta(n \log n)$ compares in the worst case. 但是注意for循环是不断插入了n个值的！所以是 $n \log n$

 B. In-place.

inplace：这个sort过程中使用了pq，耗费了额外的n空间

C. Stable.

stable：尤其关注相等键值的元素，sort前后的相对顺序是否一定保持一致

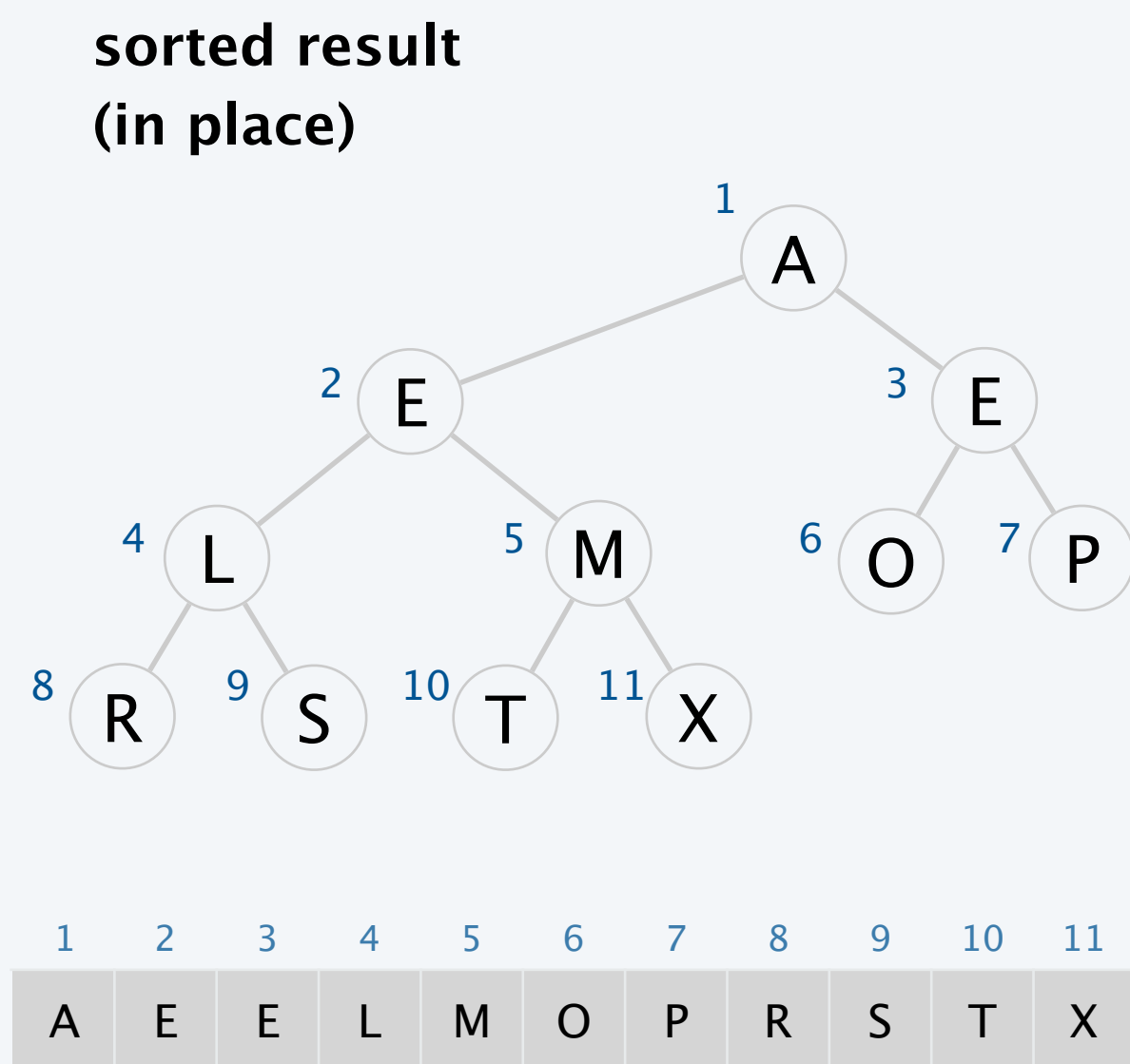
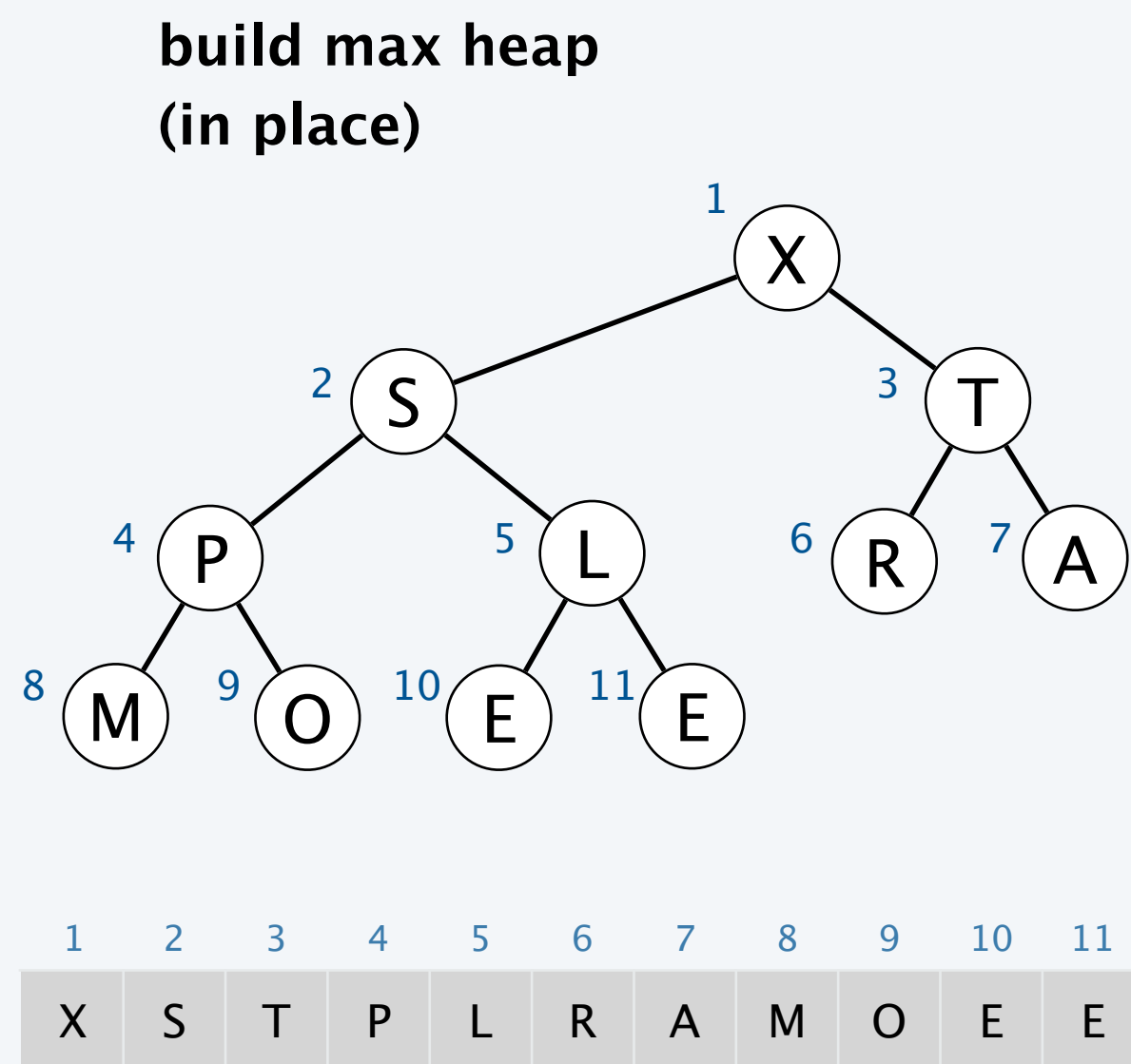
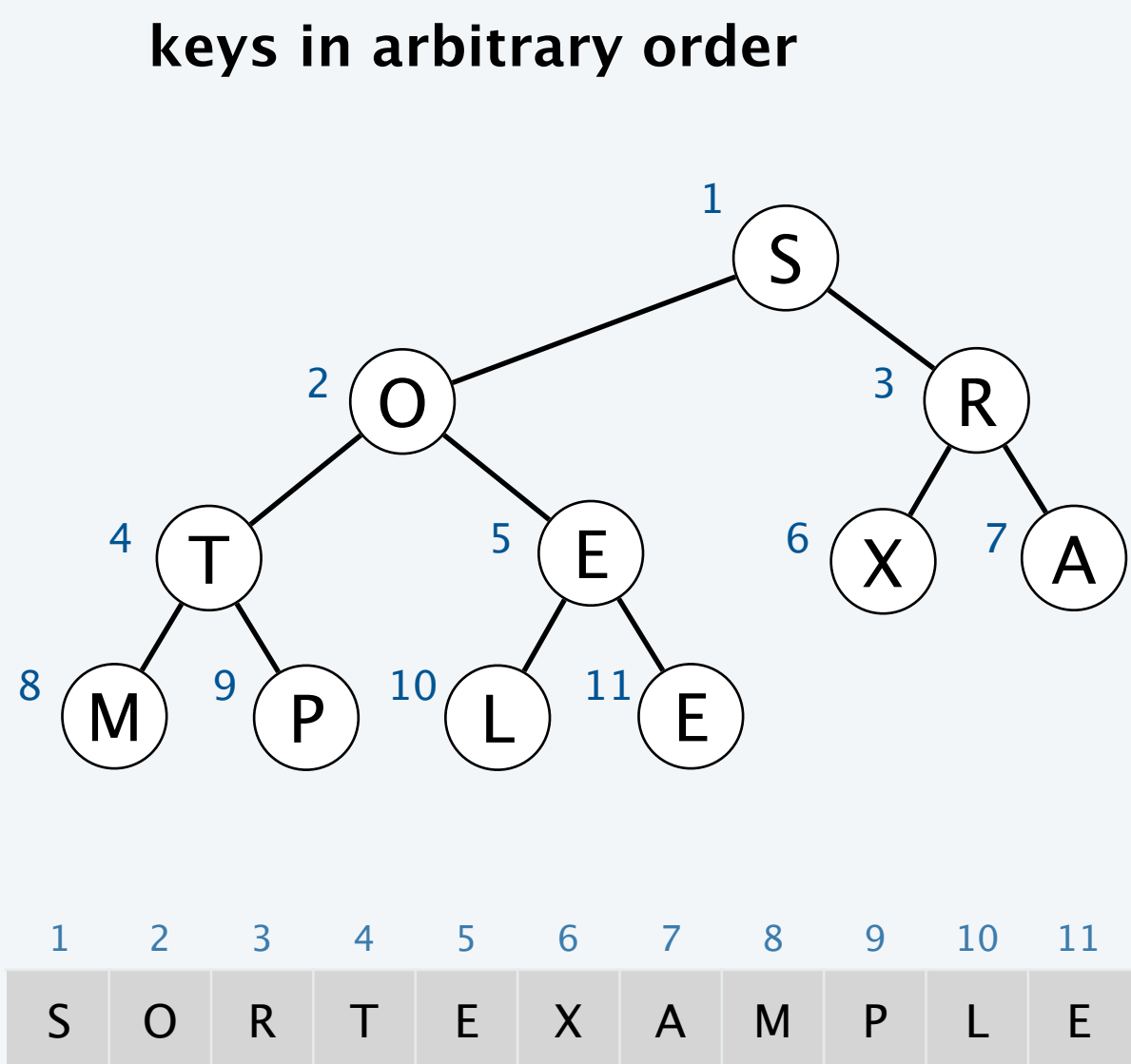
D. *All of the above.*

在标准的 MinPQ 实现中，当插入两个键值相等的元素时，后插入的元素可能会因为 swim 操作，被移动到堆的更上层（或者更下层）。

Heapsort

Basic plan for in-place sort.

- View input array as a complete binary tree. *← we'll assume 1-indexed for now*
- Phase 1 (heap construction): build a **max-oriented** heap.
- Phase 2 (sortdown): repeatedly remove the maximum key. *← a version of selection sort*

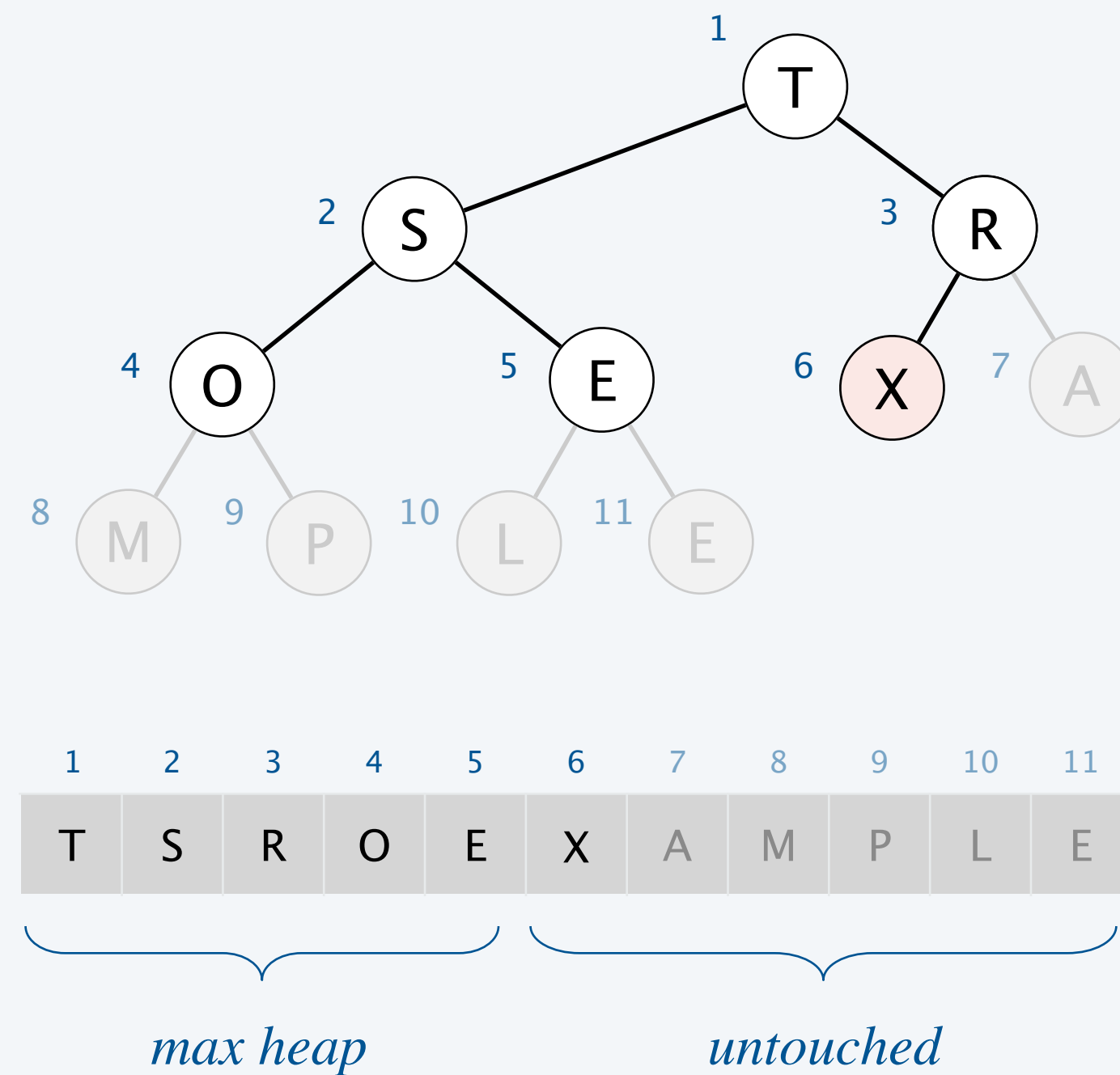


Heapsort: top-down heap construction

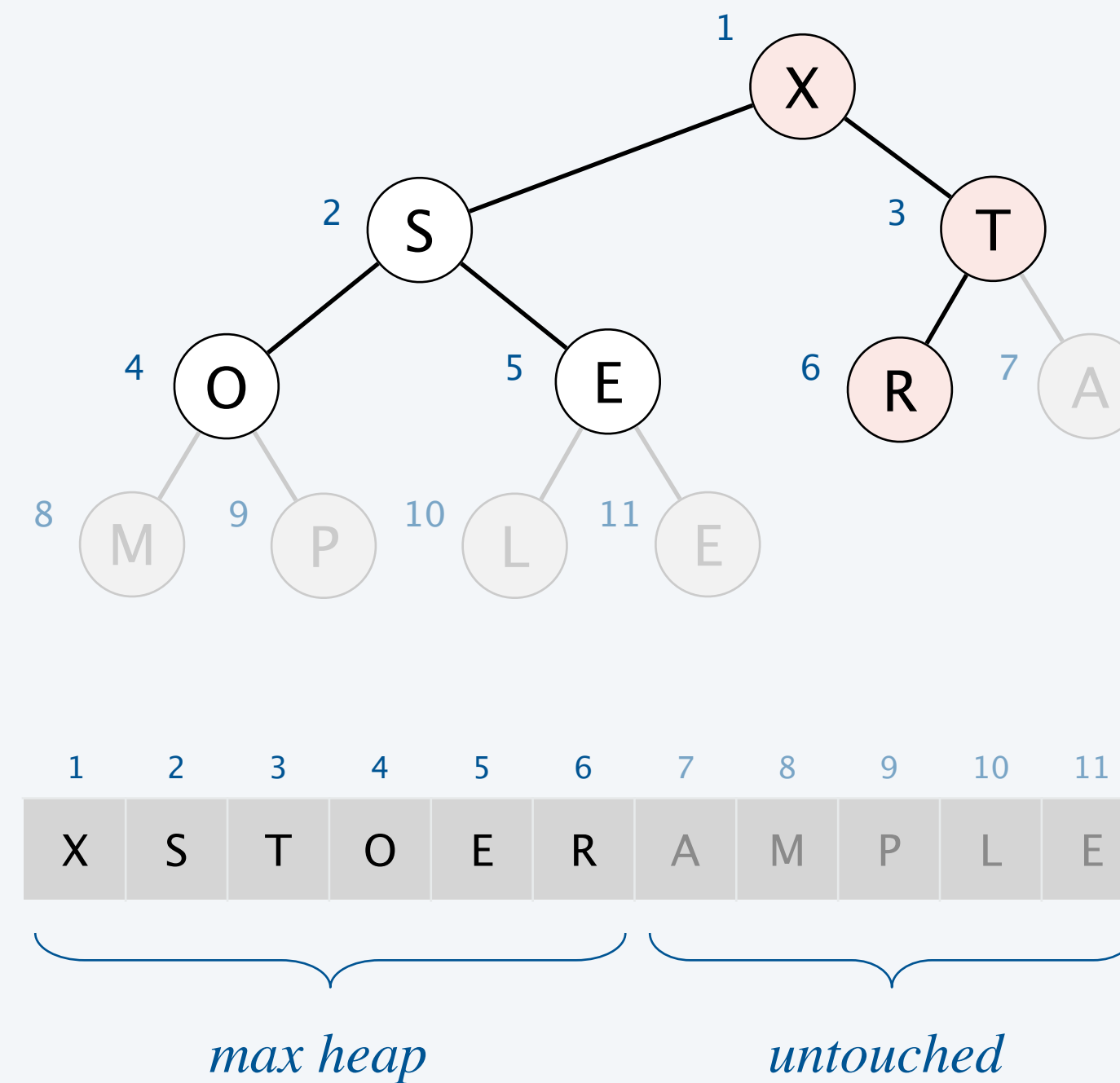
Phase 1 (top-down heap construction).

- View input array as complete binary tree.
- Insert keys into a max heap, one at a time.

before inserting X



after inserting X

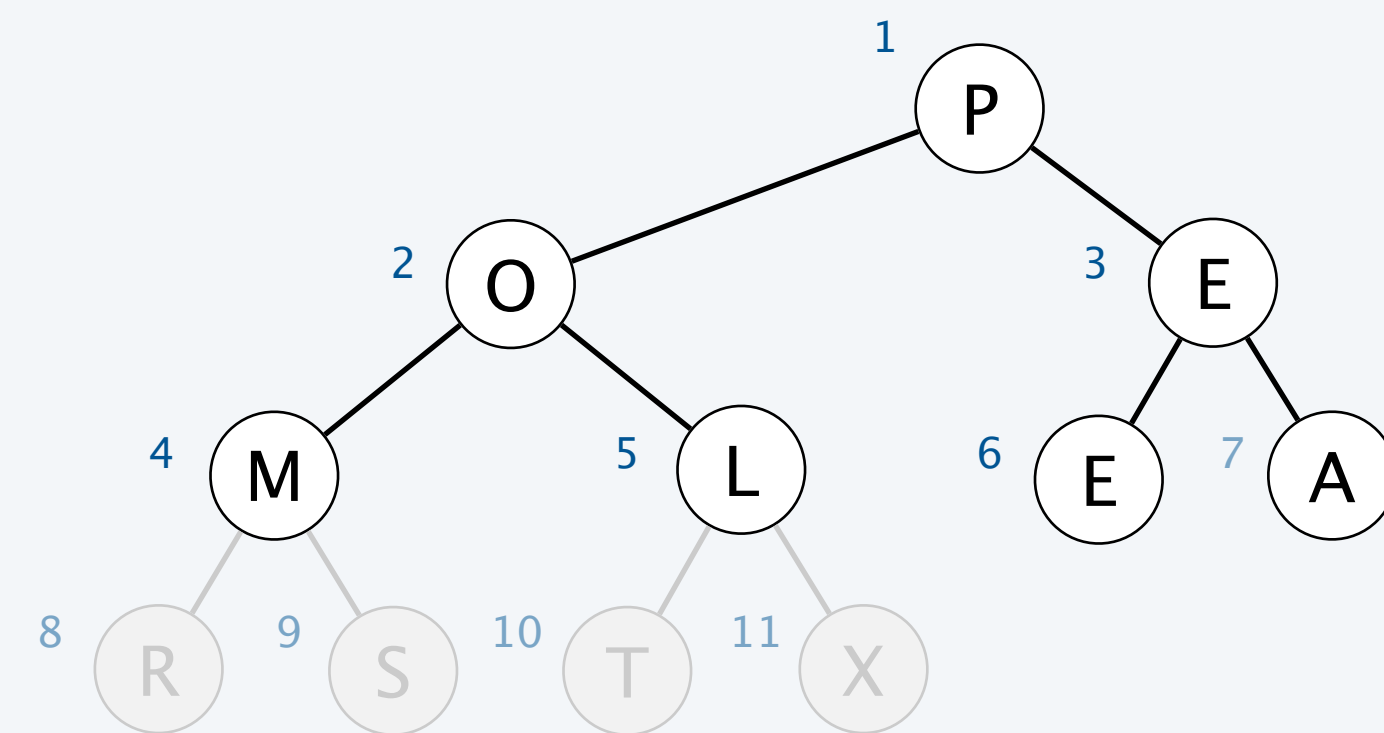


Heapsort: sortdown

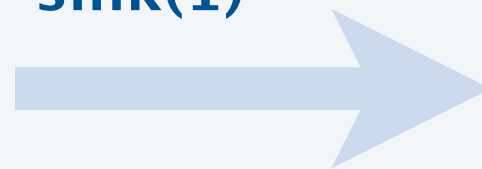
Phase 2 (sortdown).

- Remove the maximum, one at a time.
- Leave in array (instead of nulling out).

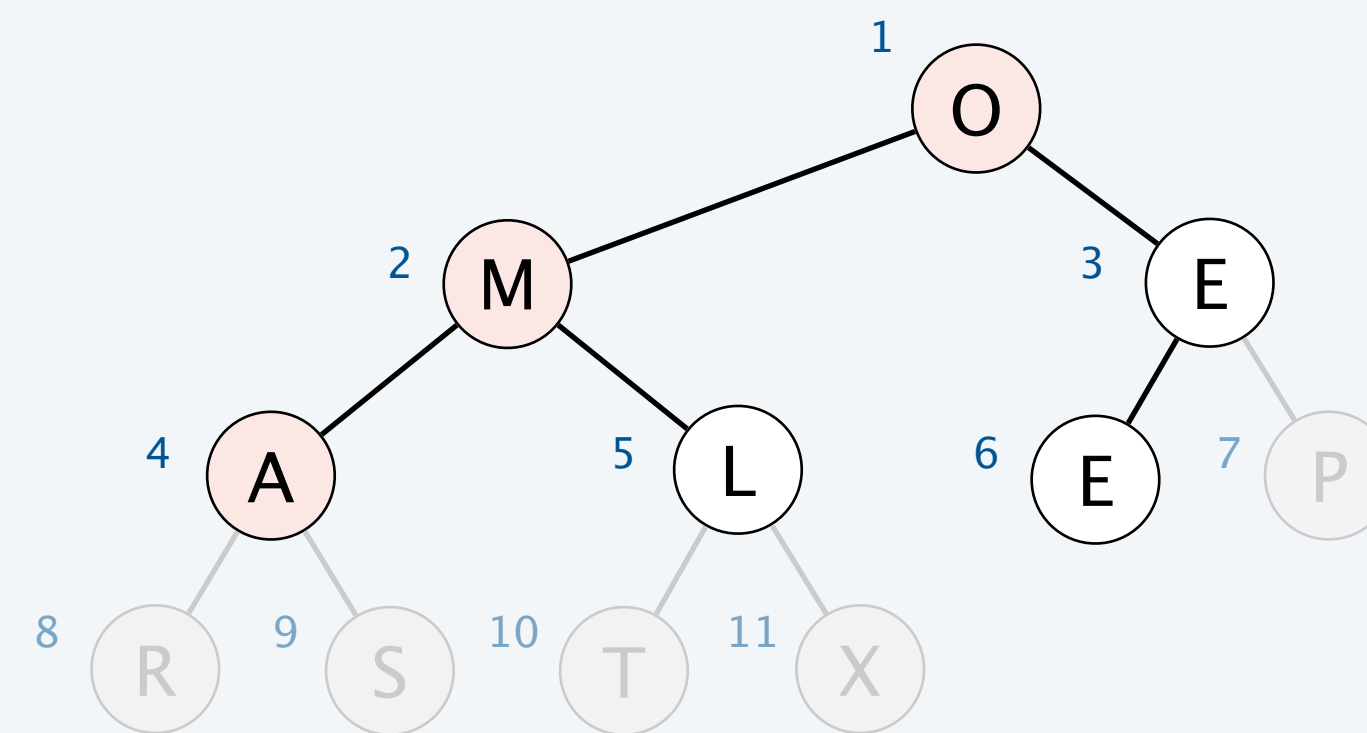
before deleting P



exch(1, 7)
sink(1)



after deleting P



Heapsort: Java implementation

```
public class HeapTopDown {  
  
    public static void sort(Comparable[] a) {  
  
        // top-down heap construction  
        int n = a.length;  
        for (int k = 1; k <= n; k++)  
            swim(a, k);  
  
        // sortdown  
        int k = n;  
        while (k > 1) {  
            exch(a, 1, k--);  
            sink(a, 1, k);  
        }  
  
        ...  
    }  
}
```

```
private static void sink(Comparable[] a, int k, int n)  
{ /* as before */ }  
  
private static void swim(Comparable[] a, int k)  
{ /* as before */ }  
  
private static boolean less(Comparable[] a, int i, int j)  
{ /* as before */ }  
  
private static void exch(Object[] a, int i, int j)  
{ /* as before */ }
```

*but make static
(and pass arguments a[] and n)*

*but convert from 1-based
indexing to 0-base indexing*

<https://algs4.cs.princeton.edu/24pq/HeapTopDown.java.html>

Heapsort: mathematical analysis

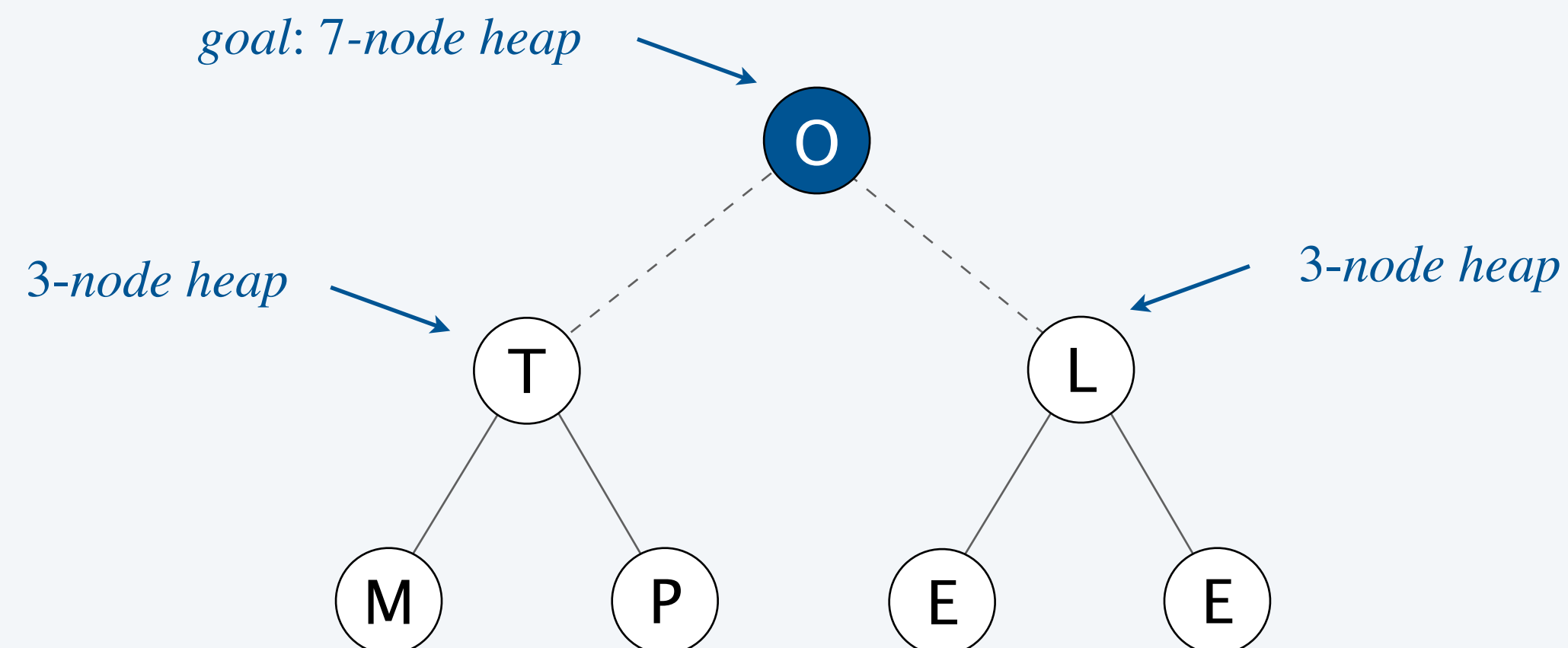
Proposition. Heapsort uses only $\Theta(1)$ extra space.

Proposition. Heapsort makes $\leq 3 n \log_2 n$ compares (and $\leq 2 n \log_2 n$ exchanges).

- Top-down heap construction: $\leq n \log_2 n$ compares (and exchanges).
- Sortdown: $\leq 2n \log_2 n$ compares (and $\leq n \log_2 n$ exchanges).

Bottom-up heap construction. [see book] Successively building larger heap from smaller ones.

Proposition. Makes $\leq 2 n$ compares (and $\leq n$ exchanges).



Heapsort: context

Significance. In-place sorting algorithm with $\Theta(n \log n)$ worst-case.

- Mergesort: no, $\Theta(n)$ extra space. $\leftarrow$ *in-place merge possible; not practical*
- Quicksort: no, $\Theta(n^2)$ time in worst case. $\leftarrow$ *$\Theta(n \log n)$ worst-case quicksort possible; not practical*
- Heapsort: yes!

Bottom line. Heapsort is optimal for both time and space, **but**:

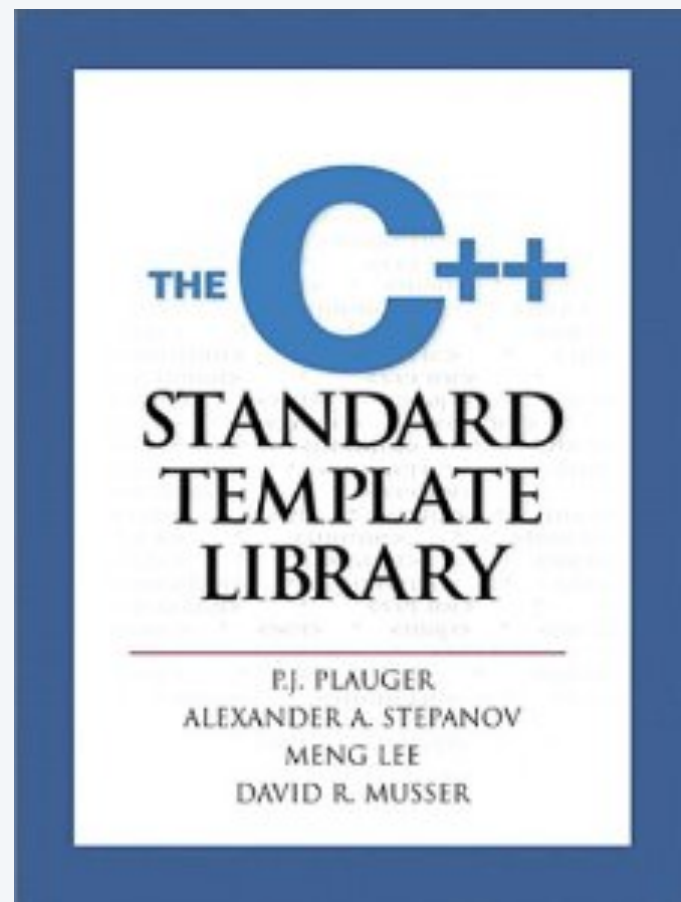
- Inner loop longer than quicksort's.
- Not stable.

Introsort

Goal. As fast as quicksort in practice; in place; $\Theta(n \log n)$ worst case.

Introsort.

- Run quicksort.
- Cutoff to heapsort if function-call stack depth exceeds $2 \log_2 n$.
- Cutoff to insertion sort for $n \leq 16$.



In the wild. C++ STL, Microsoft .NET Framework, Go.

Sorting algorithms: summary

	inplace?	stable?	best	average	worst	remarks
selection	✓		$\frac{1}{2} n^2$	$\frac{1}{2} n^2$	$\frac{1}{2} n^2$	n exchanges
insertion	✓	✓	n	$\frac{1}{4} n^2$	$\frac{1}{2} n^2$	use for small n or partially ordered
merge		✓	$\frac{1}{2} n \log_2 n$	$n \log_2 n$	$n \log_2 n$	$\Theta(n \log n)$ guarantee; stable
timsort		✓	n	$n \log_2 n$	$n \log_2 n$	improves mergesort when pre-existing order
quick	✓		$n \log_2 n$	$2 n \ln n$	$\frac{1}{2} n^2$	$\Theta(n \log n)$ probabilistic guarantee; fastest in practice
3-way quick	✓		n	$2 n \ln n$	$\frac{1}{2} n^2$	improves quicksort when duplicate keys
heap	✓		$3 n$	$2 n \log_2 n$	$2 n \log_2 n$	$\Theta(n \log n)$ guarantee; in-place
?	✓	✓	n	$n \log_2 n$	$n \log_2 n$	holy sorting grail

number of compares to sort an array of n elements

Credits

image	source	license
<i>Emergency Room Triage</i>	unknown	
<i>Car GPS</i>	<u>Adobe Stock</u>	<u>Education License</u>
<i>Complete Binary Tree</i>	<u>Shlomit Pinter</u>	by author
<i>Computer and Supercomputer</i>	<u>New York Times</u>	

A final thought

